

Gate-Based Quantum Algorithms for Combinatorial Optimisation: Simulating QAOA for the Bin Packing Problem

Guillaume Amann

<https://github.com/Guillaume-amann/Quantum>

Keywords: Quantum computing, Quantum Approximate Optimisation Algorithm, Quadratic Unconstrained Binary Optimisation, bin packing problem, gate-based quantum computing, combinatorial optimisation.

Abstract

Quantum computing is emerging as a groundbreaking paradigm with the potential to solve certain classes of problems exponentially faster than classical methods. This paper explores the application of gate-based quantum computing, specifically the Quantum Approximate Optimisation Algorithm (QAOA), to the combinatorial bin packing problem. After introducing fundamental concepts such as qubits, quantum gates, and entanglement, the bin packing challenge is formulated as a Quadratic Unconstrained Binary Optimisation (QUBO) problem and QAOA is implemented on a simulated quantum system. Utilising a parallelised MPI environment, an exhaustive parameter grid search identifies optimal variational parameters, revealing a clear energy minimum and confirming the algorithm's ability to amplify the target solution state. Limitations arising from problem size and simulation constraints are discussed, and future directions for scaling QAOA to more complex and entangled problem instances are proposed, advancing the potential for near-term quantum advantage.

Quantum Computing

Quantum computing represents a transformative leap in computational technology, characterised by two predominant paradigms: Quantum Annealing and the Gate-Based Approach. D-Wave Quantum, pioneering the field, was the first company to commercialise a functional quantum computer in 2011. Their early commitment to quantum annealing since 1999, however, has seen limited adoption by other industry players. Instead, the broader trend within the quantum computing landscape seems to have gravitated towards the gate-based approach, driven by the ambitious goal of developing the first universal quantum computer.

In this competitive and rapidly evolving sector, various companies are exploring different technological pathways. Amazon has aligned with Alice & Bob in advancing the Cat-Qubit approach, while Pasqal, Google, and IBM are focusing their

efforts on transmons-based systems. Meanwhile, ORCD, Quix or Quandela have carved a niche in optical quantum computing. Despite these diverse technological bets, one consensus emerges: the gate-based logic exhibits a versatile adaptability across multiple types of quantum computers.

Quantum computing is rapidly proving its potential as a transformative technology, with recent breakthroughs demonstrating clear advantages over classical systems. China's Jiuzhang photonic quantum computer solved a complex Gaussian boson-sampling problem in just minutes (a task estimated to take classical supercomputers billions of years) showcasing quantum supremacy in a specialised setting. Meanwhile, D-Wave's latest annealer tackled a challenging materials simulation in minutes, where classical methods would need millennia. These examples highlight how quantum systems can dramatically accelerate solutions to certain classes of problems. Motivated by this potential,

this paper explores how quantum computing, particularly gate-based logic, can be leveraged to address combinatorial challenges such as the bin packing problem.

Definition of a Qubit

Any qubit can be described as a linear combination of two orthonormal basis states, typically denoted $|0\rangle$ and $|1\rangle$. In this standard (computational) basis, the state of a qubit is described as in Equation 1, where α and β are complex probability amplitudes satisfying $|\alpha|^2 + |\beta|^2 = 1$. It is worth noting that this representation is not unique. Qubit states can indeed be expressed in different bases depending on the context or measurement axis.

To better visualise the state of a qubit, it is often useful to represent it on the Bloch sphere (Figure 1), a geometric tool where any pure state corresponds to a point on the surface of a unit sphere. In this framework, the state of a single qubit can be fully defined (up to a global phase) if and only if one knows the angles θ and φ on the Bloch sphere.

A general pure state of a qubit can be written as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1)$$

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{-i\varphi}\sin\left(\frac{\theta}{2}\right)|1\rangle \quad (2)$$

with $\theta \in [0, \pi]$ the *polar angle*
and $\varphi \in [0, 2\pi]$ the *azimuthal angle*

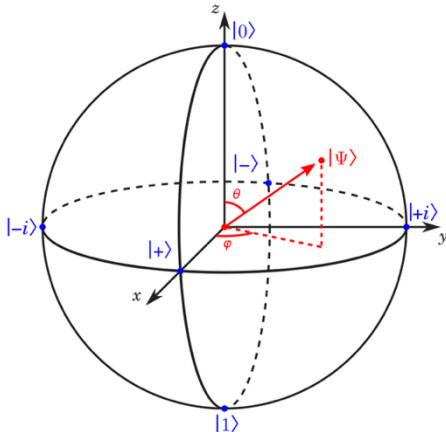


Figure 1. Bloch sphere representation of a qubit

Knowing θ and φ gives all the physically meaningful information about the pure state of the qubit. In matrix format this is presented as :

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

$$\langle\psi| = \psi^\dagger = \overline{|\psi\rangle}^T = (\bar{\alpha} \quad \bar{\beta})$$

We define ρ a Hermitian matrix representing the state as an operator. The term Hermitian means that the density matrix is equal to its own conjugate transpose, i.e., $\rho = \rho^\dagger$.

$$\rho = |\psi\rangle\langle\psi| = \psi \cdot \psi^\dagger = \begin{pmatrix} \alpha\bar{\alpha} & \alpha\bar{\beta} \\ \beta\bar{\alpha} & \beta\bar{\beta} \end{pmatrix} \quad (3)$$

$$\rho \in \mathcal{M}_2(\mathbb{C})$$

The density matrix is relevant in that its diagonal terms explicitly give us:

- $\alpha\bar{\alpha} = |\alpha|^2 = \mathbb{P}(|\psi\rangle = |0\rangle)$
- $\beta\bar{\beta} = |\beta|^2 = \mathbb{P}(|\psi\rangle = |1\rangle)$

the probabilities of a quantum system being in each basis state, making it especially useful for describing mixed states or systems undergoing decoherence. Unlike pure state vectors, which only represent ideal, isolated systems, the density matrix allows us to model statistical ensembles and partial measurements. It provides a complete description of both classical and quantum uncertainty, enabling the analysis of systems of qubits, noise, and measurement outcomes in a consistent framework.

Definition of a system

Quantum computing becomes truly powerful when working with multiple qubits, as their combined state space grows exponentially and enables phenomena like entanglement and quantum parallelism. By performing the tensor product (or Kronecker product) of two qubit state vectors, the resulting vector is a 4-dimensional complex vector representing the joint state of the two qubits. This joint system may describe either

separable states, where the qubits behave independently, or entangled states, where their behaviour becomes correlated in ways that cannot be explained classically. Understanding this distinction is fundamental when analysing systems of qubits, as it determines how information is distributed and processed across the quantum register.

➤ Two separate Qubits

When combining multiple qubits into a single system, the state space is constructed by taking the tensor product, also called the Kronecker product, of their individual state vectors.

When two qubits are in separate pure states $|\psi_1\rangle$ and $|\psi_2\rangle$, their joint (unentangled) state is described via Equation 4:

$$\begin{aligned} |\psi_1\rangle &= \begin{pmatrix} \alpha_1 \\ \beta_1 \end{pmatrix} \in \mathcal{M}_{2,1}(\mathbb{C}) \\ |\psi_2\rangle &= \begin{pmatrix} \alpha_2 \\ \beta_2 \end{pmatrix} \in \mathcal{M}_{2,1}(\mathbb{C}) \\ |\Psi_{1,2}\rangle &= |\psi_1\rangle \otimes |\psi_2\rangle = \begin{pmatrix} \alpha_1\alpha_2 \\ \alpha_1\beta_2 \\ \beta_1\alpha_2 \\ \beta_1\beta_2 \end{pmatrix} \quad (4) \\ |\Psi_{1,2}\rangle &\in \mathcal{M}_{4,1}(\mathbb{C}) \end{aligned}$$

This $|\Psi_{1,2}\rangle$ is a 4×1 column vector. No matrix, no density matrix is involved (yet), and no extra parameters are needed, but since

$$\begin{aligned} \langle \Psi_{1,2} | &= (\overline{\alpha_1\alpha_2} \quad \overline{\alpha_1\beta_2} \quad \overline{\beta_1\alpha_2} \quad \overline{\beta_1\beta_2}) \\ &= \Psi_{1,2}^\dagger \in \mathcal{M}_{1,4}(\mathbb{C}) \end{aligned}$$

can be combined with Equation 4 to adapt Equation 3, it is possible to determine the density matrix of a two separate qubits system via:

$$\begin{aligned} \rho_{1,2} &= |\Psi_{1,2}\rangle \langle \Psi_{1,2}| \\ &= \Psi_{1,2} \cdot \Psi_{1,2}^\dagger \end{aligned}$$

So, the full 2-qubit density matrix is:

$$\begin{pmatrix} |\alpha_1\alpha_2|^2 & \alpha_1\alpha_2\overline{\alpha_1\beta_2} & \alpha_1\alpha_2\overline{\beta_1\alpha_2} & \alpha_1\alpha_2\overline{\beta_1\beta_2} \\ \alpha_1\beta_2\overline{\alpha_1\alpha_2} & |\alpha_1\beta_2|^2 & \alpha_1\beta_2\overline{\beta_1\alpha_2} & \alpha_1\beta_2\overline{\beta_1\beta_2} \\ \beta_1\alpha_2\overline{\alpha_1\alpha_2} & \beta_1\alpha_2\overline{\alpha_1\beta_2} & |\beta_1\alpha_2|^2 & \beta_1\alpha_2\overline{\beta_1\beta_2} \\ \beta_1\beta_2\overline{\alpha_1\alpha_2} & \beta_1\beta_2\overline{\alpha_1\beta_2} & \beta_1\beta_2\overline{\beta_1\alpha_2} & |\beta_1\beta_2|^2 \end{pmatrix}$$

Which can also be calculated via:

$$\begin{aligned} \rho_{1,2} &= \rho_1 \otimes \rho_2 \quad (5) \\ &= (|\psi_1\rangle\langle\psi_1|) \otimes (|\psi_2\rangle\langle\psi_2|) \\ &= (\psi_1\psi_1^\dagger) \otimes (\psi_2\psi_2^\dagger) \\ &= \begin{pmatrix} \alpha_1\overline{\alpha_1} & \alpha_1\overline{\beta_1} \\ \beta_1\overline{\alpha_1} & \beta_1\overline{\beta_1} \end{pmatrix} \otimes \begin{pmatrix} \alpha_2\overline{\alpha_2} & \alpha_2\overline{\beta_2} \\ \beta_2\overline{\alpha_2} & \beta_2\overline{\beta_2} \end{pmatrix} \\ \rho &\in \mathcal{M}_4(\mathbb{C}) \end{aligned}$$

In quantum computing, the Kronecker product is used to define the joint state space of multiple qubits. In separable cases, the state vector itself can be written as a Kronecker product; in entangled systems, it cannot. Outer product is used for density matrix of two separate qubit states.

However, as mentioned earlier, not all multi-qubit systems can be described this way. Some exhibit correlations so strong that the system must be treated as a whole: this is entanglement.

➤ Entanglement

Some states exhibit correlations so strong that the state of each qubit is fundamentally linked to the others. These are known as entangled states, and they represent one of the most counterintuitive features of quantum mechanics.

While a single qubit can be fully described by two complex amplitudes (or, equivalently, a point on the Bloch sphere), the situation becomes more subtle when dealing with multiple qubits. In particular, entanglement arises when the state of a composite system cannot be written as a simple tensor product of the states of its subsystems. This represents one of the most profound departures of quantum mechanics from classical intuition as entanglement goes deeper than just synchronised

state changes. It is a quantum phenomenon where the state of a system of qubits cannot be written as a product of the states of the individual qubits. Entanglement implies the whole system must be described globally and cannot be factored into individual qubit states not even probabilistically in the pure case. Mathematically, this means:

$$\nexists f: E \rightarrow E : |\Psi\rangle = f(\psi_1, \psi_2)$$

If this condition holds, the qubits are entangled, and the quantum state of the 2-qubit system can be described as follow:

$$|\Psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$$

And its density matrix is given by:

$$\begin{aligned} \rho &= |\psi\rangle\langle\psi| = \psi \cdot \psi^\dagger \\ &= \begin{pmatrix} |\alpha|^2 & \alpha\bar{\beta} & \alpha\bar{\gamma} & \alpha\bar{\delta} \\ \beta\bar{\alpha} & |\beta|^2 & \beta\bar{\gamma} & \beta\bar{\delta} \\ \gamma\bar{\alpha} & \gamma\bar{\beta} & |\gamma|^2 & \gamma\bar{\delta} \\ \delta\bar{\alpha} & \delta\bar{\beta} & \delta\bar{\gamma} & |\delta|^2 \end{pmatrix} \end{aligned}$$

Finally, because it is possible to entangle more than 2 qubits, here is the general formulation of an n-qubit entangled quantum state $|\Psi\rangle$:

$$|\Psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i \cdot |i\rangle \quad (6)$$

with $|i\rangle$ the 2^n combinations of basis states $|00 \dots 0\rangle, |00 \dots 1\rangle, \dots, |11 \dots 1\rangle$

$$\text{and } \alpha_i \in \mathbb{C} : \sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1$$

Having established the mathematical representation of both separable and entangled quantum states, the next step is to introduce the dynamic evolution of these systems. Quantum computing is not limited to static state descriptions but fundamentally relies on the application of quantum gates that drive the evolution of states through unitary transformations. Quantum gates are unitary transformations that evolve the state over time. They may act locally on individual

qubits or globally across multiple qubits, creating entanglement and changing the system's density matrix.

Applying quantum gates

Quantum gates are the building blocks of quantum computation, analogous to logic gates in classical circuits. However, unlike their classical counterparts, quantum gates operate on continuous complex amplitudes, often acting on superpositions and entangled qubits in the process. These gates are represented by unitary matrices that preserve the total probability of the system and can act on individual qubits or across multiple qubits simultaneously. Their action transforms the state vector of a qubit, or a system of qubits, via matrix multiplication, altering both the amplitudes and phase relationships within the state. Some gates, such as the Hadamard gate, create superpositions; others, like the controlled NOT (CNOT), are essential for entanglement. Studying how these gates affect quantum states provides critical insight into quantum dynamics and is a *sine qua non* condition for building optimisation algorithms.

➤ General matrix representation of gates

A quantum gate is a unitary matrix $U \in \mathcal{M}_{2n}(\mathbb{C})$, where n is the number of qubits the gate acts on. For a single qubit, this matrix is of size 2, and the gate acts on the state vector $|\Psi\rangle$ as:

$$|\Psi'\rangle = U \cdot |\Psi\rangle$$

To preserve the total probability, quantum gates must be unitary, meaning:

$$U^\dagger \cdot U = U \cdot U^\dagger = I$$

For a multi-qubit system with no entanglement, the state of the system can be described as a Kronecker product of the states of individual qubits:

$$U = U_1 \otimes U_2 \otimes \dots \otimes U_n \quad (7)$$

In quantum computing, multi-qubit gates are fundamental. While single-qubit gates operate on

one qubit and can be represented as 2x2 matrices, multi-qubit gates operate on multiple qubits and are represented by larger matrices.

The Kronecker product can be used to construct multi-qubit gates from single-qubit gates (Equation 7), but multi-qubit gates like the CNOT gate involve interactions between the qubits that cannot be captured by a simple Kronecker product of single-qubit gates. These gates are essential for creating entanglement between qubits, which is a crucial resource in quantum computing. For a density matrix ρ , the action of a gate is given by:

$$\rho' = U \cdot \rho \cdot U^\dagger$$

This preserves both trace and Hermitian properties and is key to simulating quantum dynamics using density matrices.

➤ The Hadamard Gate

One of the most fundamental single-qubit gates is the Hadamard gate, which creates an equal superposition of $|0\rangle$ and $|1\rangle$ via the matrix described *infra*:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

The Hadamard gate is crucial because it transforms a qubit from a definite state into a superposition where both possibilities coexist with equal probability. This superposition is not just a random mix, it's a coherent quantum state where interference can occur.

This ability to initialise qubits into a balanced superposition forms the starting point of many quantum algorithms. It allows the quantum computer to process a vast number of possible solutions in parallel, with their probability amplitudes evolving and interfering through subsequent gate operations. In optimisation problems, for example, the Hadamard gate spreads out the search across all possible solutions before the circuit begins to bias the state toward more favourable outcomes using cost-encoding operations. Without such a gate, the algorithm

would lose this essential “global awareness” of the solution space that makes one key advantage of quantum computing.

➤ Rotation Gates and the Pauli Matrices

In addition to fixed gates like the Hadamard or CNOT, quantum computing relies heavily on rotation gates, which allow for continuous transformations of a qubit's state around the Bloch sphere. These gates are essential in both circuit construction and algorithm design, especially when encoding cost functions or navigating the state space during optimisation.

A general rotation around an axis of the Bloch sphere $\sigma \in \{X, Y, Z\}$ by angle θ is given by:

$$R_\sigma(\theta) = e^{-i\frac{\theta}{2}\sigma}$$

The explicit matrix forms of these gates are:

- Rotation around X:

$$R_X(\theta) = \begin{pmatrix} \cos \theta/2 & -i \sin \theta/2 \\ -i \sin \theta/2 & \cos \theta/2 \end{pmatrix}$$

- Rotation around Y:

$$R_Y(\theta) = \begin{pmatrix} \cos \theta/2 & -\sin \theta/2 \\ \sin \theta/2 & \cos \theta/2 \end{pmatrix}$$

- Rotation around Z:

$$R_Z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

These gates generalise the Pauli matrices, which are simply rotation gates with fixed angles, $X = R_X(\pi)$, $Y = R_Y(\pi)$, and $Z = R_Z(\pi)$. The latter represent 180° rotations and form the building blocks of most quantum circuits. These gates enable fine-grained control over quantum states, from precise manipulations in variational algorithms to the construction of complex unitary dynamics.

By combining gate operations, one can encode a cost function into a quantum state's phase or amplitude. This lays the groundwork for quantum optimisation algorithms such as Quantum Approximate Optimisation Algorithm (QAOA),

where sequences of gates are carefully chosen to evolve an initial superposition into a state that encodes high-quality solutions. The next section introduces how optimisation problems, like bin packing, can be formulated as Binary Quadratic Models (BQM) or Quadratic Unconstrained Binary Optimisation (QUBO) problems, which quantum algorithms can target.

Optimisation Problems

A Binary Quadratic Model (BQM) is a mathematical model used to represent problems where decision variables x_i take binary values, and the objective is to minimise a quadratic function of those variables. Formally, a BQM is expressed as:

$$E(x) = \sum_i h_i x_i + \sum_{\{i < j\}} J_{ij} x_i x_j \quad (8)$$

where the parameters h_i are the linear coefficients associated with individual variables (they represent the cost, or weight, of including variable,) and J_{ij} are the quadratic coefficients associated with pairs of variables which captures the cost or interaction when two decision variables are set to 1.

But because they typically do not handle hard constraints, the latter are instead encoded in the objective function via Lagrange multiplier. This is a key distinction from constrained optimisation in classical solvers (e.g., linear programming), where constraints are separate and enforced directly Equation 8 hence becomes:

$$E(x) = \sum_i h_i x_i + \sum_{\{i < j\}} J_{ij} x_i x_j + \lambda \cdot p(x) \quad (9)$$

with $p(x)$ a function that is 0 when constraints are satisfied, and > 0 otherwise

and λ a large coefficient (penalty weight) that makes violating the constraint “costly”

This strategy of embedding constraints directly into the energy landscape highlights the flexibility

of BQMs, especially in settings where conventional constraint handling is impractical. It also reflects a broader shift in optimisation approaches toward formulations that unify structure and objective within a single energy function.

With this foundation in place, attention now turns to a particularly important subclass of BQMs: Quadratic Unconstrained Binary Optimisation (QUBO) problems.

➤ QUBO

Quadratic Unconstrained Binary Optimisation (QUBO) is a specific formulation of binary optimisation where the entire problem, both objectives and constraints, is encoded as a single quadratic function over binary variables. Formally equivalent to the BQM, the QUBO model has become a standard in many modern optimisation frameworks due to its simplicity and versatility. Crucially, it is the native input format for quantum annealers such as those developed by D-Wave, making it a central tool in the field of quantum optimisation. This compatibility has driven widespread adoption of QUBO in both theoretical research and real-world applications involving combinatorial optimisation. The energy function is defined as follow:

$$E(x) = x^T \cdot Q \cdot x \quad (10)$$

where Q is a real-valued symmetric matrix containing both linear (diagonal terms called biases) and quadratic (off-diagonal terms called couplers) coefficients (h_i and J_{ij} from BQM formulation). If the original problem is a maximisation, it is simply negated to transform it into a minimisation, as quantum annealers are designed to seek the ground (lowest energy) state.

The cost Hamiltonian H_C is the quantum formulation of an optimisation problem, where the classical objective function (Equation 9 or Equation 10) is mapped onto a quantum operator so that its ground state (lowest energy eigenstate)

corresponds to the optimal solution of the original problem.

To make an objective function and its constraints "QUBO appropriate", the function must be a minimisation function, and constraints are typically transformed into penalty terms that equal zero when the constraint is satisfied and are strictly positive otherwise.

The cost Hamiltonian H_C , or quantum equivalent of the energy (or objective) function, for a QUBO problem is constructed by mapping binary variables for the problem $x_i \in \{0,1\}$ to quantum observables via the transformation $x_i = \frac{1-Z_i}{2}$, where Z_i is the Pauli-Z operator. The resulting Hamiltonian acts diagonally in the computational basis and takes the form:

$$H_C = \sum_i Q_{ii} x_i + \sum_{i<j} Q_{ij} x_i x_j$$

$$H_C = \sum_i Q_{ii} \frac{1-Z_i}{2} + \sum_{i<j} Q_{ij} \frac{1-Z_i-Z_j+Z_i Z_j}{4}$$

which takes the canonical form:

$$H_C = \sum_i \alpha_i Z_i + \sum_{i<j} \beta_{ij} Z_i Z_j + cte \quad (11)$$

α_i and β_{ij} are derived by substituting the QUBO variables into Pauli-Z space and collecting coefficients on linear and quadratic terms.

with a constant energy shift (do not affect the optimisation landscape, since they merely shift the energy spectrum without altering the location of the minimum).

The Equation 11 derived from a QUBO problem provides a structured quantum operator whose ground state encodes the optimal solution. While quantum annealing directly exploits this structure through adiabatic evolution, gate-based quantum computing approach can also leverage this formulation. The Quantum Approximate Optimisation Algorithm (QAOA) stands out as a

variational approach that alternates between applying the cost Hamiltonian and a mixing Hamiltonian to progressively guide a quantum system toward optimal or near-optimal solutions.

➤ QAOA

In gate-based quantum computing, QUBO formulations are not used *per se*; instead, they are generally first translated into a cost Hamiltonian H_C which encodes the objective function into the energy of quantum states (Equation 12).

In this context, the energy of the system, the Hamiltonian, and the objective function are closely related concepts and can often be used interchangeably when discussing the optimisation landscape.

$$\mathbb{E}(|\Psi\rangle) = \langle \Psi | H | \Psi \rangle \quad (12)$$

To find the ground state $|\Psi^*\rangle$, which corresponds to the optimal solution, variational quantum algorithms are typically used. The circuit prepares a quantum state $|\Psi\rangle$, measures the expected value of the Hamiltonian H (or energy of the system). The variational method approximate ground state $|\Psi^*\rangle$ and corresponding lowest energy. A classical optimiser adjusts the parameters to minimise energy. This hybrid loop continues until convergence.

$$|\Psi^*\rangle = \arg \min_{|\Psi\rangle \in \mathcal{H}} \mathbb{E}(|\Psi\rangle) \quad (13)$$

One variational quantum algorithms typically used for this kind of problem is the Variational Quantum Eigen solver (VQE). Originally developed for estimating molecular ground-state energies in quantum chemistry, VQE combines a parameterised quantum circuit, also called an *ansatz* $|\Psi(\theta)\rangle$, with a classical optimiser. The *ansatz* is set up via $U(\theta) : |\Psi(\theta)\rangle = U(\theta).|\Psi_0\rangle$ and the optimisation determines :

$$\theta^* = \arg \min_{\theta} (\mathbb{E}(|\Psi(\theta)\rangle))$$

Built on the same principle, the Quantum Approximate Optimisation Algorithm (QAOA) is a variational algorithm tailored specifically for

combinatorial optimisation problems. It prepares a parameterised quantum circuit and iteratively optimises it (with classical feedback) to approximate the ground state of H_C . QAOA emulates the effect of quantum annealing within a gate-based framework by alternating controlled unitary evolutions under two Hamiltonians: the cost Hamiltonian H_C , which encodes the objective function, and the mixer Hamiltonian H_M , which promotes exploration of the solution space by driving transitions between basis states.

The algorithm begins by specifying the problem-dependent cost Hamiltonian H_C , followed by the definition of a suitable mixer Hamiltonian H_M typically chosen to be the transverse-field mixer, $H_M = \sum_i X_i$, where X_i is the Pauli-X operator on qubit i . From these, we define two families of parameterised unitary operators: the cost unitary U_C , and the mixer unitary U_M ; defined *infra*.

$$U_C(\gamma) = e^{-i\gamma H_C}$$

applies a phase shift to each basis state that is proportional to its cost. This does not change the probability of measuring state $|0\rangle$ or $|1\rangle$, but modifies the phase φ of each component in the quantum superposition depending on how “good” or “bad” the solution is, according to the objective function.

$$U_M(\alpha) = e^{-i\alpha H_M}$$

on the other hand, is responsible for reshuffling amplitudes between different basis states. It introduces transitions between bitstrings, effectively allowing the system to explore the solution space by moving between configurations.

The unitaries are applied sequentially, and each layer k involves a pair of parameters (γ_k, α_k) that control the respective operations. A setting of $p = 1$ corresponds to a single such pair, while any layer p will lead to a total of $2p$ parameters to optimise.

Increasing the value of p enhances the expressiveness of the quantum *ansatz*, improving its ability to approximate the true ground state of

the cost Hamiltonian. This improvement can yield better optimisation outcomes, particularly for more complex problem instances. However, deeper circuits also introduce greater computational demands, both in quantum gate operations and in the classical optimisation required to tune the parameters. For small-scale systems such as a 2-qubit QUBO, low values of p remain effective and manageable, making them suitable for initial implementation and experimentation.

Next, an initial quantum state $|\Psi\rangle$ is prepared, often chosen as the uniform superposition $|+\rangle^{\otimes n}$, which ensures that all possible solutions have equal amplitude at the start. The core mechanism of QAOA involves evolving this initial state through a parameterised quantum circuit consisting of alternating p layers of parameterised unitary operators (cost and mixer operators). The depth p QAOA controls how expressively the algorithm can explore and exploit the solution space. With each layer alternating between cost-based phase shifts and amplitude mixing, the quantum state is progressively refined to concentrate probability on lower-cost solutions. QAOA thus balances structure and exploration, guiding the system toward optimal solutions through interference. Equation 13.1 constructs QAOA *ansatz* (Equation 13.2) for depth p :

$$U(p, \gamma, \alpha) = \prod_{i=1}^p U_C(\gamma_i) U_M(\alpha_i) \quad (13.1)$$

$$|\Psi_p(\gamma, \alpha)\rangle = U(p, \gamma, \alpha). |\Psi_0\rangle \quad (13.2)$$

The final goal is to find the optimal pair of parameter vectors that minimises the expectation value of the cost Hamiltonian. This expected value is estimated through repeated measurements on the quantum device, and the parameters are updated by a classical optimiser in a hybrid loop until convergence toward a low-energy (ideally ground-state) solution.

$$\mathbb{E}(\gamma, \alpha) = \langle \Psi_p | H_C | \Psi_p \rangle$$

Having established the foundational principles of quantum computing focus now turns to the question of implementation. This involves encoding quantum states, operators, and circuit dynamics into classical data structures and numerically emulating their evolution. What follows explores how to represent and manipulate quantum systems programmatically using classical computing resources.

Method

While access to large-scale quantum hardware remains limited, several cloud-based platforms now offer remote access to small- and medium-scale quantum processors. Nonetheless, many quantum algorithm prototypes continue to be developed and validated via classical simulation, particularly when customisation or low-level control is required. In this work, a programming-based approach is adopted to reconstruct core components of quantum computation from first principles, without reliance on external quantum libraries. Qubits, quantum gates, circuit evolution, and variational algorithms such as the Quantum Approximate Optimisation Algorithm (QAOA) are implemented manually in C++, enabling full transparency over the simulation process and allowing direct experimentation with the underlying mechanics of quantum optimisation.

➤ Problem formulation

Before constructing the quantum circuit, the optimisation problem must be precisely defined in classical terms. The chosen problem is a minimal bin packing instance, formulated as a QUBO problem to allow direct translation into quantum form. The system consists of a 2×1 bin, i.e., two independent cells. An empty cell incurs a penalty of +0.3, while a filled cell yields a reward of -0.5. Let $x_i \in \{0, 1\}$ denote the binary variable associated with cell i , where $x_i = 1$ indicates a filled cell. The classical cost function C is:

$$C = 0.3(1 - x_0) + 0.3(1 - x_1) - 0.5x_0 - 0.5x_1$$

which simplifies to:

$$C(x_0, x_1) = -0.8x_0 - 0.8x_1 + 0.6$$

Because there are no constraints and no interaction between the variables, the problem requires only two qubits with no entanglement. The objective is to minimise C , which will later be mapped into a quantum cost Hamiltonian.

The quantum simulation is run using $n = 2$ qubits, with a parameter grid search of size 1000×1000 over $\gamma, \alpha \in [0, 2\pi[$. For each parameter pair, the expectation value of the cost Hamiltonian is computed, and the minimum energy state determines the optimal parameters (γ^*, α^*) . The optimal state is then measured $S = 1000$ times in the computational basis to obtain the distribution over classical bitstrings $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$.

➤ Implementing QAOA

The classical cost function C previously derived contains a constant term of +0.6, which can be discarded for the purposes of QAOA implementation, as explained earlier. The relevant objective therefore reduces to:

$$\min -0.8x_0 - 0.8x_1$$

In order to express this objective in quantum mechanical form, each binary variable $x_i \in \{0, 1\}$ is mapped to a Pauli-Z operator using the standard transformation $x_i = \frac{1 - Z_i}{2}$, where Z_i acts on qubit i . Substituting this into the reduced cost function yields

$$-0.8x_i = -0.8 \frac{1 - Z_i}{2} = -0.4 + 0.4Z_i,$$

so that the overall cost Hamiltonian H_C becomes:

$$H_C = 0.4Z_0 + 0.4Z_1$$

This diagonal operator assigns an energy penalty proportional to the spin alignment along the Z-axis, thus encoding the objective function in quantum terms. To enable exploration of the search space, the Quantum Approximate Optimisation Algorithm employs a mixer

Hamiltonian. For unconstrained binary optimisation, the standard choice is the transverse-field operator:

$$H_M = X_0 + X_1$$

where X_i is the Pauli-X operator acting on qubit i . This Hamiltonian generates bit-flip dynamics, allowing superposition states to evolve across different computational basis states during the optimisation process.

The QAOA procedure begins with the uniform superposition state $|+\rangle^{\otimes n}$, obtained by applying Hadamard gates to both qubits. For a single QAOA layer ($p = 1$), the variational *ansatz* is prepared by applying first the cost Hamiltonian unitary $e^{-i\gamma.H_C}$, followed by the mixer Hamiltonian unitary $e^{-i\alpha.H_M}$, where γ and α are tunable parameters. The present work evaluates the cost function expectation value over a dense 1000×1000 grid in the parameter domain $(\gamma, \alpha) \in [0, 2\pi]$. This exhaustive search yields the energy surface of the problem, from which the optimal parameter pair (γ^*, α^*) corresponding to the minimum expectation value is identified.

Once the ground-state parameters are found, the corresponding quantum circuit is executed, and the output state is measured 1000 times in the computational basis. The relative frequencies of the outcomes $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ are then recorded, producing a measurement histogram that reveals the dominant bitstring(s) associated with the optimal solution. This measurement distribution provides a direct classical interpretation of the quantum solution, enabling comparison with the known optimum from the classical formulation.

➤ Simulating quantum behaviour

In this quantum simulation framework, the `Qbit` and `Gate` classes work in tandem to represent and manipulate quantum states through linear algebraic operations (see Figure 2) (note there is no misspelling here but a deliberate chose of the author to call a qubit a `Qbit`). A `Qbit` object

models the state of an n -qubit register that can be described, as shown earlier, as a complex-valued state vector of length 2^n . It provides constructors for either creating a register in the default computational basis state $|0 \dots 0\rangle$ or initialising it from an arbitrary state vector, ensuring normalisation. The class offers core functionality for quantum computation: a `measure` method simulates projective measurement by probabilistically selecting a basis state according to the squared amplitudes, collapsing the state vector to the observed basis state; an `apply` method multiplies the current state vector by the unitary matrix of a `Gate`, thereby evolving the system according to the gate's definition; and a `print_state` method outputs the non-zero components of the state in Dirac notation.

The `Gate` class represents quantum gates as unitary matrices acting on n qubits, internally stored as a two-dimensional vector of complex numbers. It can be constructed either from a specified number of qubits, producing a zero-initialised matrix of the appropriate dimension, or from an explicit matrix, in which case the dimension is validated to be a power of two, and the number of qubits is inferred. The class provides a variety of static factory methods for common quantum gates (Pauli-X, Pauli-Y, Pauli-Z, Hadamard, and single-qubit rotations around the X, Y, and Z axes), as well as methods to construct multi-qubit gates through tensor products (or Kronecker) and to embed single-qubit gates into larger systems, via `expand`. An identity gate generator is also included. Last but not least, operator overloading allows matrix multiplication between two gates, facilitating the composition of quantum operations.

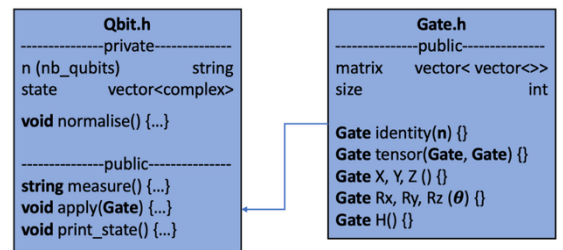


Figure 2. Class diagram of the Quantum Simulator

Interaction between the two classes occurs primarily through the `Qbit::apply` method, which takes a `Gate` as input, checks that the gate’s dimension matches the state vector’s dimension and performs the matrix–vector multiplication to yield the new quantum state. This design cleanly separates concerns: `Gate` encapsulates the mathematical description of quantum operations, while `Qbit` maintains and evolves the state of the quantum register. Together, they form a minimal but extensible infrastructure for simulating quantum circuits, allowing complex gates to be built from primitives and applied sequentially to model quantum algorithms.

Results and limitations

The simulation was executed on a parallel computing environment using MPI to perform a full two-dimensional grid search over the parameter space $\gamma, \alpha \in [0, 2\pi[$, discretised into 1000×1000 evaluation points. For each parameter pair, the QAOA circuit was prepared, evolved according to the cost and mixer Hamiltonians, and its energy expectation value computed. This exhaustive computation enabled the construction of the complete energy surface, from which the global minimum could be visually identified.

The resulting surface (Figure 3) exhibited a single, well-defined basin corresponding to the lowest-energy configuration. MPI-based aggregation identified the optimal parameters with $\gamma \approx 1.962$ and $\alpha \approx 5.497$, yielding a minimum energy of $E_{min} \approx -0.799$. These parameters were then broadcast to all processes for the measurement phase.

Using the optimal parameters, the state was measured 1000 times, with measurement responsibilities distributed evenly among the MPI processes and later reduced to rank 0. The resulting histogram (Figure 4) showed an unambiguous dominance of the $|11\rangle$ state, consistent with the trivial structure of the cost Hamiltonian, which

favours configurations in which both qubits take the “1” value. The relative frequency of $|11\rangle$ exceeded that of all other outcomes by a large margin, confirming that the QAOA procedure, even at this basic depth, identified and amplified the target solution state.

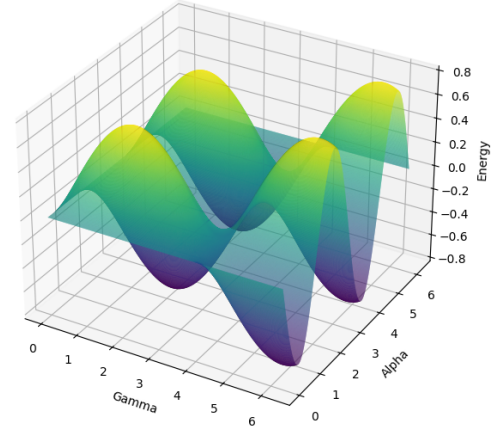


Figure 3. Energy surface of the system

The parallel execution times were also recorded, with the parameter grid search constituting the overwhelming majority of runtime, and the measurement phase accounting for only a small fraction. The ability to generate the complete energy landscape not only confirmed the optimal parameter location but also served as a validation step against possible local minima or degenerate solutions.

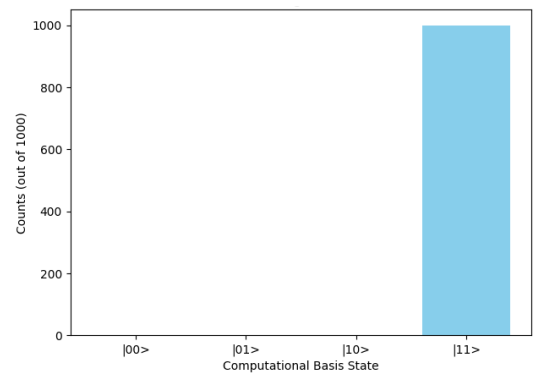


Figure 4. Measurement histogram of Ground State

➤ Limitations

While the present implementation successfully demonstrates the end-to-end QAOA workflow, it is constrained by several simplifying factors. The cost Hamiltonian used here is intentionally elementary, resulting in an energy landscape with

a clear and easily identifiable global minimum. This simplicity, while beneficial for validation purposes, limits the generality of the conclusions regarding QAOA's performance on more complex or degenerate optimisation problems.

When designing a QAOA algorithm, it is important to avoid overly simple or symmetric cost Hamiltonians and to manage coefficient scaling carefully. A symmetric cost function, such as one based solely on individual Pauli-Z terms, often results in a flat energy landscape where many parameter combinations yield similar energies. This flatness masks meaningful structure in the parameter space and complicates the identification of truly optimal parameters.

Another key limitation is the exhaustive grid search approach used to locate the optimal parameters. Although this method yields a complete energy surface and a reliable global minimum identification in the current small-scale problem, it does not scale well to larger qubit counts or higher QAOA depths. As problem complexity grows, more sophisticated parameter optimisation techniques will be necessary to maintain computational feasibility.

Finally, it is worth emphasising that the simulation is entirely classical and idealised, lacking noise and hardware imperfections inherent in real quantum devices. Consequently, the measurement statistics demonstrating the clear supremacy of the $|11\rangle$ solution reflect a best-case scenario.

➤ Future Work

Future research will focus on extending the QAOA framework to more complex problem instances that require genuine entanglement and involve hard constraints, such as combinatorial optimisation problems with multi-qubit interactions. Incorporating interaction terms and nontrivial coupling will yield richer energy landscapes that more accurately represent practical challenges. Additionally, exploring advanced parameter optimisation methods and noise-resilient implementations will be critical for

scaling the algorithm beyond toy examples and toward near-term quantum advantage.

Conclusion

The application of gate-based quantum algorithms, particularly QAOA, to combinatorial optimisation problems such as bin packing has been demonstrated. Detailed simulation on a parallel computing framework enabled identification of optimal variational parameters and verification of algorithm convergence toward the expected solution. Limitations in simulation scale and noise resilience currently restrict practical deployment, yet the results highlight the promise of quantum computing to accelerate complex optimisation tasks beyond classical capabilities. Future research will focus on incorporating multi-qubit interactions, advanced parameter optimisation techniques, and noise mitigation strategies to extend the approach to larger, more realistic problem domains.